

Министерство науки и высшего образования Российской
Федерации
Южный федеральный университет
Физический факультет

КУРСОВОЙ ПРОЕКТ

по дисциплине «Администрирование ОС Linux»

Тема: Использование NFS-сервера для сетевого хранения данных

Авторы (Группа):

■■■■■■■■■■ ■. ■.
■■■■■■■■■■ ■. ■.
■■■■■■■■■■ ■. ■.

Проверил:

■■■■■■■■■■ ■■■■■ ■■■■■■■■■■

Ростов-на-Дону, 2026

Содержание

Введение

1 Распределение ролей в группе и индивидуальный вклад

1.1 Инженер по автоматизации инфраструктуры — Григорян А. А.

1.2 Системный администратор / SRE — Карлашов А. В.

1.3 Инженер по мониторингу и безопасности — Кроливец И. С.

2 Теоретические основы технологии NFS

3 Проектирование архитектуры и сетевая изоляция

4 Автоматизация развертывания и концепция IaC

4.1 Декларативное описание стека

4.2 Автоматизация жизненного цикла окружения

5 Настройка NFS-сервера и клиентов

6 Системная безопасность и hardening Linux-окружения

7 Тестирование, резервное копирование и CI/CD

Заключение

Введение

Современный этап развития корпоративной ИТ-инфраструктуры предъявляет всё более высокие требования к централизованному хранению и совместному использованию файловых ресурсов. В средах с большим числом узлов — будь то физические серверы, виртуальные машины или контейнеры — каждый узел не должен хранить собственную копию данных; необходима единая точка хранения, доступная всем участникам инфраструктуры одновременно.

Протокол NFS (Network File System), разработанный компанией Sun Microsystems в 1984 году и прошедший эволюцию вплоть до версии NFSv4.2, остаётся отраслевым стандартом сетевых файловых систем для Unix-подобных сред. Его нативная интеграция с ядром Linux, высокая производительность при работе с большими файлами и богатые возможности разграничения прав доступа делают NFS оптимальным выбором для задач горизонтального масштабирования хранилищ и организации совместной работы контейнеров с общими данными.

Однако ручная настройка NFS-сервера неизбежно порождает «дрейф конфигураций» (configuration drift) и ошибки воспроизводимости. Применение методологии Infrastructure as Code (IaC) совместно с инструментами контейнеризации Docker позволяет полностью автоматизировать жизненный цикл развёртывания, устранив человеческий фактор.

Цель данного проекта — комплексное проектирование, развёртывание, настройка и защита NFS-сервера в контейнеризированной среде под управлением ОС Linux, а также обеспечение безопасного сетевого доступа к нему для множества клиентских контейнеров.

1 Распределение ролей в группе и индивидуальный вклад

Разработка, тестирование и обеспечение безопасности сетевого файлового хранилища выполнялись проектной группой из трёх человек с чётким разграничением обязанностей и зон инженерной ответственности.

Участник	Роль	Основные задачи
Григорян Ашот	Backend Engineer / SRE	Docker Compose, bootstrap.sh, Dockerfile, CI/CD
Карлашов Андрей	Backend Engineer / SRE	MySQL, Redis, Kafka, Elasticsearch, Prometheus, Grafana, healthcheck, CI/CD
Кроливец Иван	Backend Engineer / SRE	UFW/firewall hardening, Prometheus + Grafana, CI/CD

1.1 Инженер по автоматизации инфраструктуры — Григорян Ашот

В зону ответственности инженера по автоматизации входила разработка декларативного описания всей инфраструктуры и автоматизация её жизненного цикла:

- Проектирование структуры Git-репозитория и настройка ветвления (main, feature/*, gh-pages);
- Разработка главного оркестрационного манифеста docker-compose.yml с описанием NFS-сервера, клиентских контейнеров и изолированных виртуальных сетей;
- Написание скрипта bootstrap.sh первичной подготовки ОС: установка зависимостей, создание директорий экспорта, настройка прав доступа;
- Сборка оптимизированных Dockerfile с применением принципа наименьших привилегий (non-root user) и многоэтапной сборки (multi-stage builds);
- Автоматизация управления стендом через Makefile: цели up, down, test, clean.

1.2 Системный администратор / SRE — Карлашов Андрей

Системный администратор сфокусировался на конфигурации сервисного уровня и обеспечении отказоустойчивости:

- Настройка NFS-сервера (nfs-kernel-server / NFS Ganesha): конфигурация файла /etc/exports, опций rw, sync, root_squash, fsid;
- Конфигурация нескольких клиентских контейнеров с автоматическим монтированием NFS-шары через Docker Volume (driver: local, type: nfs);
- Написание и интеграция скриптов healthcheck для проверки доступности NFS-экспорта;
- Проведение HA-тестов: имитация падения сервера, замер времени восстановления доступа к данным клиентами;
- Проверка блокировок файлов при одновременной записи из нескольких контейнеров.

1.3 Инженер по мониторингу и безопасности — Кроливец Иван

Инженер по безопасности обеспечивал многоуровневую защиту хранилища и наблюдаемость системы:

- Разработка скрипта hardening.sh: конфигурация UFW по принципу «запрещено всё, кроме явно разрешённого»; открыты только порты SSH (22), NFS (2049) для доверенных подсетей;
- Изоляция сети: NFS-трафик строго ограничен виртуальной сетью storage-net, внешний доступ к порту 2049 заблокирован;
- Развёртывание стека мониторинга Prometheus + Grafana с экспортером node_exporter и cAdvisor для сбора метрик контейнеров;
- Разработка дашбордов Grafana: нагрузка на NFS-шару, I/O дисковой подсистемы, состояние контейнеров;
- Настройка конвейера CI/CD в GitHub Actions: линтеры ShellCheck, yamllint, Hadolint; блокировка слияния при провале тестов.

2 Теоретические основы технологии NFS

NFS (Network File System) — клиент-серверный протокол, позволяющий монтировать удалённые файловые системы как локальные. В ядре Linux реализована поддержка NFSv3 и NFSv4/4.1/4.2. Архитектура NFS строится на модели RPC (Remote Procedure Call) с транспортом по протоколам TCP и UDP.

Версия NFSv4 принципиально отличается от предшественников: она использует единственный порт 2049/TCP, обеспечивает интегрированную безопасность через Kerberos или RPCSEC_GSS, поддерживает сессионную модель с управлением состоянием соединения и атомарные операции аренды (lease). NFSv4.1 добавляет параллельное расположение данных (pNFS), а NFSv4.2 — серверную копию файлов без участия клиента (server-side copy).

Ключевые параметры экспорта NFS, задаваемые в файле `/etc/exports`:

- `rw / ro` — разрешение записи или только чтения для клиента;
- `sync` — гарантия записи данных на диск до подтверждения клиенту (важно для целостности данных);
- `async` — более высокая производительность, но риск потери данных при сбое;
- `root_squash` — перемаппинг `root`-пользователя клиента в анонимный `UID/GID` (безопасность);
- `no_root_squash` — отключение маппинга (используется только в доверенных средах);
- `fsid=0` — обязателен для `NFSv4`, определяет корень псевдофайловой системы экспорта.

Сравнение NFS с альтернативными решениями

При выборе технологии хранения была проведена сравнительная оценка трёх подходов:

Технология	Преимущества	Недостатки	Вывод
NFS	• Простота настройки и использования • Высокая производительность Linux, macOS, FreeBSD, Solaris, OpenSolaris, FreeBSD, OpenSolaris, FreeBSD, OpenSolaris	• Необходимость настройки файлового сервера • Ограничения в поддержке файловой системы • Проблемы с безопасностью	Оптimalен для задачи
Samba (SMB)	• Поддержка протоколов SMB 2.0 и 3.0 • Интеграция с Active Directory AD/LDAP	• Более сложная настройка • Зависимость от библиотеки Samba Linux-библиотека, Samba-сервер • Зависимость от Docker	• Универсальность

GlusterFS	<pre> # glusterfs # glusterfs # glusterfs # glusterfs </pre>	<pre> # glusterfs # glusterfs # glusterfs # glusterfs 3-# glusterfs </pre>	<pre> # glusterfs </pre>
-----------	--	--	--------------------------

3 Проектирование архитектуры и сетевая изоляция

Архитектура стенда построена по принципу минимальной связности компонентов. Все сервисы развёрнуты в Docker-контейнерах и взаимодействуют исключительно через изолированные виртуальные сети.

Состав кластера

nfs-server — Основной сервис хранения данных. Экспортирует директорию /exports/shared по протоколу NFSv4 на порту 2049. Примонтированный именованный том nfs_data обеспечивает персистентность данных.

nfs-client-1 — Первый клиент. Автоматически монтирует NFS-шару через Docker Volume (driver: local, type: nfs4) при старте контейнера. Имитирует сервис приложения.

nfs-client-2 — Второй клиент. Монтирует ту же NFS-шару, демонстрируя совместный доступ к данным из нескольких контейнеров одновременно.

monitoring — Контейнер Prometheus + Grafana для сбора метрик нагрузки на NFS-сервер и дисковой подсистемы.

Сетевая топология

Безопасность сетевого контура реализована через виртуализацию Docker. Созданы две изолированные сети:

- storage-net (bridge) — приватная сеть для NFS-трафика. NFS-сервер и оба клиента подключены исключительно к этой сети. Внешний доступ к порту 2049 отсутствует.
- monitoring-net (bridge) — отдельная сеть для Prometheus/Grafana. Дашборды Grafana доступны на порту 3000 хост-машины с базовой HTTP-аутентификацией.

Сетевое разграничение гарантирует, что NFS-трафик (в том числе данные файлов) не может быть перехвачен другими контейнерами и не покидает виртуальный сегмент storage-net.

4 Автоматизация развёртывания и концепция IaC

Развёртывание инфраструктуры реализовано в рамках парадигмы IaC с помощью декларативного инструмента Docker Compose. Это полностью исключает дрейф конфигураций и гарантирует идентичность сред разработки, тестирования и промышленной эксплуатации.

4.1 Декларативное описание стека

Ниже приведён фрагмент конфигурационного файла `docker-compose.yml`, описывающий NFS-сервер и одного из клиентов:

```
services:
  nfs-server:
    image: erichough/nfs-server:latest
    container_name: nfs-server
    restart: always
    privileged: true
    environment:
      NFS_EXPORT_0: '/exports/shared *(rw, sync, no_subtree_check, fsid=0)'
    volumes:
      - nfs_data:/exports/shared
    networks:
      storage-net:
        ipv4_address: 172.20.0.10

  nfs-client-1:
    image: alpine:3.19
    container_name: nfs-client-1
    restart: always
    depends_on:
      nfs-server:
        condition: service_healthy
    volumes:
      - type: volume
        source: nfs_shared
        target: /mnt/shared
    networks:
      - storage-net
```

4.2 Автоматизация жизненного цикла окружения

Скрипт `bootstrap.sh` берёт на себя низкоуровневую подготовку хост-системы. Он выполняет проверку наличия файла `.env`, создаёт структуру директорий экспорта и устанавливает корректные права доступа POSIX:

```
#!/bin/bash
# @file bootstrap.sh @brief ████████████████████ NFS-██████████████████
set -euo pipefail

EXPORT_DIR="${EXPORT_DIR:-/opt/nfs/shared}"
```

```
if [[ ! -d "${EXPORT_DIR}" ]]; then
    mkdir -p "${EXPORT_DIR}"
    chown nobody:nogroup "${EXPORT_DIR}"
    chmod 777 "${EXPORT_DIR}"
    echo "LOG: ██████████ ████████████████████ ████████████████████ ${EXPORT_DIR}"
fi
```

5 Настройка NFS-сервера и клиентов

Конфигурация NFS-сервера определяется файлом `/etc/exports`, описывающим экспортируемые директории и права доступа для клиентов.

Конфигурация файла `/etc/exports`

```
/exports/shared 172.20.0.0/24(rw, sync, no_subtree_check, fsid=0, root_squash)
/exports/shared 172.20.0.11(rw, sync, no_root_squash, fsid=0)
```

Строка для подсети `172.20.0.0/24` включает `root_squash` — перемapping привилегированных запросов в анонимного пользователя, что снижает риск несанкционированного повышения привилегий. Для доверенного клиента `172.20.0.11` (тестовый контейнер) `no_root_squash` разрешает работу от `root` в изолированной среде.

Монтирование NFS через Docker Volume

Клиентские контейнеры монтируют NFS-шару напрямую через встроенный драйвер Docker Volume без установки дополнительных пакетов в образ:

```
volumes:
  nfs_shared:
    driver: local
    driver_opts:
      type: nfs4
      o: "addr=172.20.0.10,rw, sync, nfsvers=4"
      device: ":/exports/shared"
```

Контроль жизнеспособности (healthcheck)

Для NFS-сервера настроен `healthcheck`, проверяющий доступность экспорта командой `showmount`:

```
healthcheck:
  test: ["CMD", "showmount", "-e", "localhost"]
  interval: 30s
  timeout: 10s
  retries: 5
  start_period: 20s
```

6 Системная безопасность и hardening Linux-окружения

Многоуровневый периметр защиты охватывает сетевой стек ОС хоста, права доступа файловой системы и изоляцию контейнеров.

Межсетевой экран UFW

Скрипт `hardening.sh` переводит UFW в режим блокирования входящего трафика по умолчанию. Открыты только строго необходимые порты:

Порт / Протокол	Назначение	Ограничение
22/TCP	SSH — <code>ssh -o StrictHostKeyChecking=no</code>	<code>admin-IP</code>
2049/TCP	NFS — <code>storage-net</code>	<code>storage-net</code> , <code>storage-net</code>
3000/TCP	Grafana — <code>grafana</code>	<code>localhost/admin-3000</code>
9090/TCP	Prometheus — <code>prometheus</code>	<code>localhost</code>

Принцип наименьших привилегий в контейнерах

- Все контейнеры, кроме `nfs-server`, запускаются от непривилегированного пользователя (`USER appuser`);
- `nfs-server` требует `privileged: true` для работы с ядерными модулями NFS — это задокументированное исключение;
- `cap_drop: [ALL]` применяется ко всем клиентским контейнерам;
- Корневая ФС клиентов монтируется в режиме `read_only: true`.

7 Тестирование, резервное копирование и CI/CD

Стратегия интеграционного тестирования

Валидация развёрнутого стенда осуществляется скриптом `test_nfs.sh`, который проверяет три уровня работоспособности:

```
##### 1: #####: #####
##### NFS-##### (showmount -e nfs-server) ■ ##### 60
#####.

##### 2: #####: #####, ##### NFS-#####
##### ■ ##### df -h | grep nfs.

##### 3: #####: ##### nfs-client-1 ■ #####
##### nfs-client-2 — #####.

##### 4: HA-#####: ##### nfs-server, #####
#####.
```

Резервное копирование

Скрипт `backup.sh` реализует «горячее» резервное копирование содержимого NFS-экспорта без остановки сервиса. Применяется утилита `rsync` с флагами `--archive --checksum`, обеспечивающими инкрементальное копирование только изменённых файлов. Ротация архивов удаляет копии старше 7 дней:

```
find "${BACKUP_DIR}" -type f -name "*.tar.gz" -mtime +7 -exec rm {} \;
```

Конвейер CI/CD (GitHub Actions)

Проект интегрирован с облачным конвейером GitHub Actions. Пайплайн `pr-validation.yml` запускается автоматически при открытии Pull Request и включает следующие этапы:

- `yamllint` — проверка синтаксиса всех YAML-файлов проекта;
- `ShellCheck` — аудит bash-скриптов на логические ошибки и соответствие POSIX;
- `Hadolint` — проверка `Dockerfile` на оптимизацию слоёв сборки;
- Динамические тесты — развёртывание стенда в эфемерном Ubuntu-раннере и запуск `test_nfs.sh`.

Слияние кода в ветку `main` блокируется до успешного прохождения всех этапов конвейера.

Заключение

В ходе выполнения данного курсового проекта был успешно спроектирован, развёрнут и защищён отказоустойчивый сервис сетевого файлового хранилища на базе протокола NFS в контейнеризированной среде под управлением ОС Linux.

Применение методологии Infrastructure as Code и инструментов Docker Compose позволило полностью автоматизировать все этапы: подготовку окружения, настройку экспортов, монтирование в клиентских контейнерах и проведение интеграционных тестов.

Реализованная многоуровневая система безопасности — изоляция сети storage-net, правила UFW по принципу «запрещено всё», принцип наименьших привилегий в контейнерах — обеспечила высокий уровень защищённости хранимых данных от несанкционированного доступа.

Развёрнутый стек мониторинга Prometheus + Grafana с кастомными дашбордами предоставляет полную наблюдаемость (observability) работы NFS-сервера в реальном времени. Интеграция с CI/CD пайплайном GitHub Actions гарантирует стабильность инфраструктуры на протяжении всего жизненного цикла её эксплуатации.

Результат	Статус
■■■■■■■■■■■ NFS Replica Set ■ Docker Compose	✓ Выполнено
■■■■■■■■■■■ bootstrap.sh ■ Makefile	✓ Выполнено
■■■■■■■■■■■ ■■■■■■■■■■■■ (UFW, cap_drop, ■■■■)	✓ Выполнено
■■■■■■■■■■■ ■■■■■ ■ HA-■■■■■■■■■	✓ Выполнено
■■■■■■■■■■■ Prometheus + Grafana	✓ Выполнено
CI/CD ■■■■■■■■■■ GitHub Actions	✓ Выполнено